

crono**audit**

Schedule critical analysis, from the file to the verdict. How to install, how to use, how to read the report, and how to reproduce every figure.



AUTHOR
Helton da Silva de Oliveira

FORMAT
Agent Skill · Apache 2.0

EDITION
English

About this manual

This manual accompanies version 0.6.1 of cronoaudit. It was written for whoever receives a schedule and has to say whether it is true, and it does not assume the reader knows the concepts: every chapter starts from the concept, then shows what the tool does with it.

Every example uses the synthetic schedules shipped with the tool. No real contract, contractor or person appears here.

The tool follows the open Agent Skills format and works with any agent that reads it, and with no agent at all. Source, licence and the latest version are at github.com/Holiv/cronoaudit.

Contents

1	What cronoaudit does, and for whom	4.3	The views, V0 to V9
1.1	For whom	4.4	The findings
1.2	What goes in and what comes out	4.5	Conventions
1.3	The eight answers in the report	4.6	The reading layer
1.4	What it does not do	5	The nine checks
2	Concepts this manual assumes	5.1	Population and cross-cutting rules
3	Installation and first run	5.2	A1 · Out-of-sequence execution
3.1	Install with one command	5.3	A2 · Total inversion
3.2	With no assistant at all	5.4	H · Forecast elapsed with no actual
3.3	Exporting the schedule	5.5	E · Pulled forward and never started
3.4	First run	5.6	C · Delayed beyond the threshold
3.5	Inside an assistant: how to start the conversation	5.7	G · Duration inconsistent with the window
3.6	Verify the tool before trusting it	5.8	B · Milestone with no deadline
3.7	An equivalent implementation inside Project	5.9	F · In progress at zero percent
		5.10	P · Pending reporting
4	The report, section by section	6	Network quality
4.1	Cover and verdict	6.1	How to read it
4.2	Cards and index		

7	Productivity and trend by resource	10.5	The executive synthesis
7.1	Where the quantities live	11	The JSON files
7.2	Three rates, not one	11.1	Single-file review
7.3	Four dates, three verdicts	11.2	Cycle comparison
7.4	Thin evidence	11.3	Two identifiers, always together
7.5	Contracted productivity	11.4	Re-reading the report as data
8	Looking forward and looking back	12	Limits, labels and what the tool does not decide
8.1	Earned Schedule	12.1	The three labels
8.2	Look-ahead	12.2	What the tool refuses to do
8.3	Rates by group	12.3	Judgements that stay with the person
8.4	Milestone bands	12.4	Formats and scope
8.5	Rain exposure	13	Frequently asked questions
8.6	Forensics: driving path per milestone	13.1	The report flags zero on everything in a file I know has problems
8.7	Calendar forensics	13.2	cronoaudit's count does not match my macro's
8.8	Execution pattern	13.3	Earned value does not match Project
8.9	With a previous snapshot	13.4	Why is the report in English when the schedule is mine
9	Cycle comparison	13.5	Can I change the 30-day threshold
9.1	Input	13.6	E flagged activities that were only reprogrammed
9.2	Matching an activity across versions	13.7	Does sensitive data leave the machine
9.3	The three signals	13.8	How do I update
9.4	The deviation decomposition	13.9	Does it work with Primavera P6
9.5	One function, called twice	14	Glossary
9.6	What comes out		
10	Profile and customisation		
10.1	Discover before asking		
10.2	The profile keys		
10.3	Create and edit		
10.4	The two interviews in the assistant		

What cronoaudit does, and for whom

cronoaudit reads a schedule, checks whether it holds together, measures what is done against what was planned, projects what comes next, and writes a report in which every figure says where it came from and how to reproduce it. It looks for one specific kind of failure: the arithmetic is right, every term is right in its own source, the result is plausible, nothing errors, and the number means nothing.

For whom

For whoever has a schedule in front of them and has to say whether it holds: the schedule received from another party, the one produced in-house, or the one under their responsibility to update. The reader may sit in a contractor, a supervising body, a regulator, a consultancy, or be studying. The tool does not distinguish the role of whoever uses it; it distinguishes the file. The planner who has to deliver a schedule that passes, the engineer auditing what was received and the manager who needs a cycle report for the board use the same report.

The manual assumes no prior knowledge of the concepts. Chapter 2 explains each one from scratch. Readers who already have them can go straight to chapter 3.

What goes in and what comes out

INPUT	The schedule exported from MS Project as XML, which is Project's own documented format. One file for the critical review; two files, previous and current, for the cycle comparison.
OUTPUT	A self-contained HTML report with no external reference, which opens in any browser and prints to PDF. Beside it, one JSON file per analysis, for whoever wants the data outside the report.
LANGUAGE	The schedule's. The tool detects it from the content and writes the report in Portuguese or English. It can be forced.
REQUIREMENTS	Python 3.9 or later. Nothing else. No data leaves the machine.

The eight answers in the report

SECTION	QUESTION IT ANSWERS
Verdict and findings	Does the logic hold? Which dates have elapsed, moved, or are misreported?
S-curve	Where should the contract be, where is it, and does the tool's figure match Project's?
Progress by group	Did progress happen where the money is?

Productivity	At three rates, when does each activity in progress finish, and does it enter the critical path?
Network quality	Does the schedule work as a model, before any discussion of dates?
Looking forward	When does it really finish, what do the next weeks demand, and where does the pace fall short?
Forensics	Where did each milestone's delay come from, by which path, and when did it start?
Cycle comparison	What changed since the last delivery: execution, replanning, or the reference itself?

What it does not do

It does not read Primavera P6: the method transfers, but nothing was measured in that format. It does not produce a contractual delay claim: the forensics reconstruct the mechanism, and a claim needs contemporaneous records and a formal method. It does not certify against the DCMA standard: the fourteen metrics are implemented and labelled as an implementation. And it never invents a distribution, a default value or a figure the file does not carry. Chapter 12 covers this in detail.

Concepts this manual assumes

Every concept below appears in the report. If one is new, read its box before going on. If all are familiar, this chapter is a vocabulary reference.

THE SCHEDULE AS A NETWORK

A schedule is not a list of dates. It is a network: each activity has predecessors, which must happen before it, and successors, which depend on it. The link type says how: finish-to-start is the common one, the successor only starts when the predecessor finishes; start-to-start, finish-to-finish and start-to-finish are the other three. A link can carry a lag, a mandatory wait, or a lead, a permitted overlap.

When the scheduling tool recalculates dates, it walks this network. If the network says one thing and the works do another, every calculated date rests on logic that is not being followed. That is why the report checks the network before any date.

BASELINE

The baseline is a copy of the plan saved at a certain moment: dates, durations and costs as they were when the plan was approved. Everything the report calls variance, delay or pull-forward is measured against it. MS Project keeps up to eleven baselines, numbered 0 to 10, and a schedule that has grown may hold the current one in a slot that is not the default. Which slot counts is a choice, and cronoaudit makes it from the data, not from a field that declares and is not confirmed.

STATUS DATE

The date up to which progress is valid: the schedule's "today". Every comparison of planned against actual is made at it. A file without a status date cannot be analysed honestly, and cronoaudit refuses to assume today's date instead, because that would change the answer every day the analysis was repeated.

CALENDAR AND WORKING DAY

Each activity has a calendar, which says which days and hours are working time. That is where holidays, the shift, the five- or six-day week and the weather reserve live, the days the works do not work because of rain. Two activities in the same schedule can have different calendars. Every duration, variance and window in the report is counted on the activity's own calendar, because counting in calendar days, or on the project calendar, gives wrong numbers in silence.

PHYSICAL PERCENT AND PERCENT COMPLETE

They are two different fields. Percent complete is duration: how much of the activity's time has passed, and Project computes it on its own from actual dates. Physical percent is declared by a person and measures how much of the work is out. One says how much time passed; the other, how much was delivered. Earned value uses the physical one. Swapping one for the other changes the result with nothing changed on site.

EARNED VALUE AND THE S-CURVE

Earned value is each activity's baseline cost multiplied by its physical percent. Summed, it is how much of the budget has been "earned". Planned value is how much should have been earned by the status date, according to the time-phased baseline. The S-curve is the cumulative chart of both over time: planned and earned, and the gap between the curves is the delay in money. The schedule performance index, SPI, is earned divided by planned.

FLOAT AND CRITICAL PATH

An activity's total float is how much it can slip without moving the project finish. Zero float means any delay there pushes the end: the activity is critical. The critical path is the chain of critical activities from start to finish. An activity with float can slip until it consumes it; after that, it enters the critical path. Project computes float, early and late dates and the critical flag; cronoaudit reads those fields rather than recomputing them.

EARNED SCHEDULE

Classic earned value measures schedule in money, and that has a defect: at the end of the project earned always reaches the budget, so the schedule indicator goes to 1.0 even on a project a year late. Earned Schedule measures schedule in time: it asks on what date the planned curve reached the value earned today. That date, compared with the time elapsed, gives a schedule efficiency and an independent finish, which is set against the finish the schedule projects. Chapter 8 details the formulas.

PRODUCTIVITY BY RESOURCE

A material resource, such as cubic metres of cut, has planned, executed and remaining quantities per activity. Productivity is quantity per working day. Comparing the practised rate with the required one says whether the activity finishes when the schedule says. cronoaudit reads the quantities from the file's own assignments, with no custom field.

THE REFERENCE QUESTION

An indicator that compares planned with actual has two halves. If they are measured against different references, another baseline slot, another instant, another population, another unit, the difference between them means nothing, and the number comes out plausible. It is the failure cronoaudit looks for everywhere. The question that finds it: are all the terms of this calculation measured against the same reference?

Installation and first run

cronoaudit is an Agent Skill: a folder with instructions for an assistant and the programs that do the work. It installs by copying that folder to where the assistant looks for skills. The installer does that for every assistant at once.

Install with one command

On macOS or Linux, in a terminal:

```
curl -fsSL https://raw.githubusercontent.com/Holiv/cronoaudit/main/install.sh | bash
```

On Windows, in PowerShell:

```
irm https://raw.githubusercontent.com/Holiv/cronoaudit/main/install.ps1 | iex
```

The installer downloads the package, copies the skill to the two directories that, together, every assistant reads, and runs the tool's internal test as proof of installation. At the end it shows where the skill went and how to invoke it. Restart the assistant after installing, because skills are read at the start of each session.

DIRECTORY	WHO READS IT
~/.claude/skills/cronoaudit	Claude Code, OpenCode, Cursor
~/.agents/skills/cronoaudit	Codex, Gemini CLI, Cursor, GitHub Copilot, VS Code, OpenCode

Each assistant also has a native install path: `npx skills add Holiv/cronoaudit -g` in any client of the format, `gemini skills install` with the repository URL in Gemini CLI, Codex's skill installer, Cursor's Remote Rule, and the Claude Code plugin. The repository README keeps the table current.

With no assistant at all

The tool works without artificial intelligence. Clone or download the repository and run the program directly. Everything is produced except the written executive synthesis and the customisation interview, which are the two parts that depend on an assistant.

```
git clone https://github.com/Holiv/cronoaudit
python3 cronoskills/cronoaudit/scripts/review.py delivery.xml
```

Exporting the schedule

The .mpp file is a closed proprietary format. cronoskills reads the XML that Project itself exports, which is a documented format and carries everything the analysis needs: tasks, links with lag, the eleven baselines, calendars, custom fields, assignments and the time-phased data.

IN MS PROJECT

File, Save As, and under file type choose XML Format. Keep the XML beside the .mpp. When the XML is sent with the .mpp by whoever produces the schedule, this step disappears from the routine.

There is an optional path to read .mpp directly, through a Java library, for whoever has it installed. It is never required.

First run

```
python3 ~/.claude/skills/cronoaudit/scripts/review.py delivery.xml
```

On a schedule of six thousand activities the run takes under a minute. The terminal shows the summary: baseline slot used, threshold, population, counting convention, and the list of nine findings with each one's activity count. Next to the input file sit the results.

FILE	WHAT IT IS FOR
delivery-review.html	the report, to open, present and print to PDF
delivery-review.json	the findings as data
delivery-review-scurve.json	the monthly and per-group S-curve
delivery-review-productivity.json	rates by resource and by activity
delivery-review-quality.json	the fourteen metrics with their items
delivery-review-forecast.json	Earned Schedule, look-ahead, bands, rain
delivery-review-forensics.json	driving chains and execution pattern
delivery-review-readings.json	each section's rule-built readings, to quote without composing

Useful options: `--outdir` to choose where to write, `--group-by` to group weight by a custom field, `--profile` to apply the organisation profile, `--lang` to force the language, `--threshold-days` and `--tolerance-days` to move the thresholds.

Inside an assistant: how to start the conversation

With the skill installed, no command needs to be remembered. The assistant reads the skill's instructions, runs the programs, reads the results and talks about them. The figures always come from the programs; the assistant orchestrates, interprets and writes. What follows is the most direct way to begin.

Where to open

The assistant reads the local disk, so it has to know where the file is. Two equivalent ways: open the terminal in the folder that holds the XML and start the assistant there, or start anywhere and give the full path in the first message. When the folder holds more

than one XML, the message should name the file; the assistant is instructed to list the files and ask which one to use, never to choose silently.

The first message

```
cronoaudit, review the schedule delivery-september.xml
cronoaudit, analyse the schedule at ~/Schedules/2026-09/delivery.xml
cronoaudit, compare previous.xml with current.xml
cronoaudit, analyse delivery.xml and save the results in ~/Reports/september
```

The skill's name at the start of the message makes sure the assistant loads it. From there the language is free.

What happens next

The analysis runs once. It and the report are the same command: whenever the analysis runs, the HTML report and the JSON files are written beside the XML, or in the folder named in the message. The assistant states the report's full path in its first reply. After that the conversation continues over the data already written, point by point, without running again. Nothing is sent anywhere; the files stay where they were written.

The report does not have to be opened for the conversation to happen. A finding, a rate or a milestone can be discussed for an hour before the HTML is opened, or it may never be opened. The executive synthesis is written only when asked for.

Useful questions and requests

REQUEST	WHAT IT PRODUCES
save the results in ~/Reports/september	writes the report and the JSON files in the named folder, creating it if needed
group by DISCIPLINE	the progress-by-group view uses the custom field instead of the WBS
use the profile profile.json	applies the organisation's declared conventions
report in Portuguese	forces the language, ignoring detection
explain finding A1 before anything else	the assistant reads the pairs and explains each, writing no synthesis
which C activities carry the most baseline cost?	orders the finding by weight, which the report table does not
when does it finish by Earned Schedule, and what does that reading not see?	the independent finish beside the schedule's own, with the limit stated
where did the delay on the milestone in row 412 come from?	the driving chain and the origin, with row numbers
which rates are thin evidence?	separates the supported verdicts from the ones that are questions

write the executive synthesis and inject it into the report	four to six paragraphs, with author and time, from the JSON files
customise the organisation profile	the profile interview, starting with field discovery
run it again, the file changed	a new analysis over the updated XML, overwriting the results

Free plans

Without an assistant the tool is entirely free: Python and the command produce the full report with the rule-built readings. With an assistant it depends on the assistant's plan. As of this edition, Claude Code is not part of Anthropic's free plan and requires a subscription or API credit; Gemini CLI and GitHub Copilot offer free tiers that read the Agent Skills format. Plans change; check at the time.

Verify the tool before trusting it

```
python3 ~/.claude/skills/cronoaudit/scripts/test_checks.py
```

The test runs the tool over two synthetic schedules. On the positive one every check must fire. On the negative one every check must stay silent. Without the second half a clean result is worth nothing: a probe that cannot fire reports zero findings on a broken file as readily as on a sound one.

An equivalent implementation inside Project

The same method exists as macros that run inside MS Project, marking activities in the schedule itself and generating the same report. It is a separate product by the same author, and cronoaudit does not depend on it. The two implementations follow the same rule specification, and one's count has to match the other's on the same file.

The report, section by section

The report is a single HTML file. It opens in any browser, works offline, prints to PDF with one button, and switches between light and dark themes. Activity tables are collapsed, so the report does not open as a wall of rows; printing expands them.

Cover and verdict

The cover carries the file, the number of leaf activities, the status date and the baseline cost. The verdict shows cumulative physical progress weighted by cost, beside a fixed text on how to read those numbers, and the first **Reading** block: sentences generated by rule from this analysis's figures. A red notice appears when activities sit in violated logic, and says to settle that before discussing any date.

When an executive synthesis has been injected, it appears right below the cover, with the name of whoever wrote it and the time, so prose is never mistaken for measurement.

Cards and index

One card per finding, with the count and the severity colour. The index lists the views and the findings with code, title, subtitle, count and severity, and each row leads to its section.

SEVERITY	FINDINGS	READS AS
critical	A1, A2	invalidates every date below until settled
high	H, E, C, B	affects the deadline or the control of it
check	G, F	a symptom to inspect, not a finding to report
pending	P	a reporting defect, not an execution one

The views, V0 to V9

V0	S-curve from the file's own phasing: planned, earned, and the other earned-value method as sensitivity. Monthly table with cumulatives, variance and SPI. The task-by-task reconciliation against the file at the foot.
V1	Progress by group: planned against earned in each group, weighted by the group's own cost, and one column per finding. The group is the WBS branch by default or a custom field.
V2 T0 V4	Starts per month, finish variance in eight bands, total float in five bands.
V5	Calendars that carry the work: activities per calendar, shift, days per week and non-working exceptions.
V6	Productivity and trend by resource. Chapter 7.

V7	Network quality, the fourteen metrics. Chapter 6.
V8	Looking forward: Earned Schedule, look-ahead, rates by group, milestones with bands, rain exposure. Chapter 8.
V9	Forensics: driving path per milestone, calendar forensics, execution pattern. Chapter 8.

The findings

Each finding is a block with code, severity, title and plain-language subtitle. Inside, three fixed blocks: problem, impact and solution. Then the criterion with the counting convention, where it came from, with the Project columns, and how to reproduce it in the tool. Finally the activity table, collapsed. Only the visible row number identifies an activity, because it is the only identifier a person can find on screen. In the network findings the table shows the explicit pair, successor and predecessor, with the link type and the reason.

Conventions

The last section lists what the report assumed: the baseline slot and why it was chosen, the threshold and its unit, the tolerance, the population, the counting convention, how working days were counted, where the percentage comes from, the detected language and how confident the detection was. Nobody has to guess anything.

The reading layer

Every section opens with sentences generated by rule from its own figures. They say what the data supports and nothing more; if a figure is missing, the sentence does not appear. It is what lets the report be handed to someone who was not in the room. The executive synthesis is something else: a written text, injected with author and time, which chapter 10 explains.

The nine checks

Each check has a stable code, which stays the same when the implementation changes. Without that a problem cannot be discussed across weeks. The order matters: network first, then dates, then reporting.

Population and cross-cutting rules

Every check looks only at leaf activities, active and not external. Summaries stay out, because they would count their children twice. Milestones are in everything except G.

PENDING REPORTING MIGRATES FIRST

A predecessor at 100% physical with no actual finish is a reporting defect, finding P. Its links are not evaluated for A1 or A2. Without this rule, a reporting problem would be reported as an execution problem, and the conversation with whoever executes would start by accusing the wrong thing.

A1 • Out-of-sequence execution

A successor started while the predecessor had not finished, or finished after the successor's start beyond what the link allows.

PROBLEM	The network describes an order that execution does not practise.
IMPACT	Every forecast date is computed on a false premise. And the delay of stalled predecessors stops propagating to milestones, because the successors have already been delivered: the plan looks healthy at the end because it is broken in the middle.
SOLUTION	Settle before discussing any date. Either the link is wrong, or the sequence was broken on site and the consequence enters the forecast. Do not delete the link to silence the warning: it is the only record that the order was not followed.

The criterion evaluates the four link types, with actual dates, on the successor's calendar, honouring lag. An overlap covered by a lead is not flagged. Pairs with both activities complete leave the count, by decision, and are recorded in the data with the overlap in days, because the history of out-of-order execution is evidence for the forensics.

LINK	SUCCESSOR	PREDECESSOR	RESULT
FS	actual start, not complete	no actual finish	A1, predecessor not finished
FS	actual start, not complete	finished after the start, beyond the lead	A1, overlap beyond lead
FS	complete	complete	outside the count, recorded
SS	actual start	started after the lag allows	A1, before lag
FF	actual finish	finished after the lag allows	A1, before lag

SF	actual finish	started after the lag allows	A1, before lag
Both ends of the violated link are marked, and the report counts three ways: distinct activities, as successors, as predecessors. How many activities have the problem depends on how they are counted, and diverging from the convention of the other party's tool hands them the argument.			
TO REPRODUCE IN PROJECT Group by predecessor and filter by Actual Start populated. Mark the pair and inspect the link in the Gantt.			
A2 • Total inversion A successor is complete and the predecessor never started. It is the extreme form of a broken sequence, and it is unanswerable in a meeting. Either the sequence was invented for the plan, or the progress landed on the wrong activity. Check pair by pair and record which, because the answer varies between pairs.			
TO REPRODUCE Filter by Actual Finish populated and check the Actual Start of each predecessor.			
H • Forecast elapsed with no actual A forecast start is in the past with no actual start, or a forecast finish is in the past with no actual finish. The plan says it happened; nothing in the file says it happened. Every successor is projected from a date that already failed to hold. Two possibilities with different answers: the work happened and was not recorded, or it did not happen and the forecast must move. Leaving it is the only wrong option.			
E • Pulled forward and never started The forecast finish moved earlier than the baseline by 30 working days or more, on the activity's calendar, and the activity never started. Progress pulled earlier with no execution behind it is a plan change dressed as performance, and in aggregate it can offset real delay elsewhere. Ask what changed to justify the date. If nothing did, the forecast is optimistic and should go back.			
C • Delayed beyond the threshold The forecast finish moved later than the baseline by 30 working days or more, on the activity's calendar. Completed activities stay in the list on purpose: consummated delay is information. It is the finding the other party expects and the easiest to contest, which is why it needs the reproduction attached. Read it with the weight distribution: the same number of delayed activities means different things depending on the budget share they carry. The float column says how much margin is left.			

THRESHOLD IN WORKING DAYS

The 30 days are counted on the activity's own calendar, not in calendar days nor on the project shift. On a six-day week, 30 working days are about 35 calendar days. The unit is declared in the conventions section and adjustable through the profile.

G • Duration inconsistent with the window

The duration does not match the working time between start and finish, measured on the activity's own calendar, with partial first and last days. G is a thermometer, not a finding: it says something was edited inconsistently, not what. Start anchored on the actual date with the remainder pushed, a typed date over a computed one, a constraint fighting the logic. Use it to choose what to inspect, looking at the largest differences first. Never report it alone.

B • Milestone with no deadline

The milestone has no Deadline set, so nothing in the file makes its date binding. Without it the milestone shows comfortable float even with the contractual date passed, because the schedule was never told the date mattered. A deadline is not a constraint: it moves nothing, it only reveals slippage. Register contractual dates on the milestones that have them; it changes no calculation and makes every following report tell the truth about float. B is optional by default, because not every organisation practises it; the profile promotes it to a finding.

F • In progress at zero percent

Actual start, no actual finish, and duration percent complete at zero. On a schedule reported by physical percent this fires a lot, because the duration percent is not maintained; that is why it is treated as a check. On a schedule reported by duration, it is the real signal of a stalled activity.

P • Pending reporting

One hundred percent physical with no actual finish date. It is a reporting defect, and the distinction decides who the conversation is with. Kept among the network findings, it would accuse the team of executing out of sequence when the work was done and only the date is missing. Enter the actual finish. Until then the activity stays deliberately out of the network findings.

Network quality

Does this schedule hold as a model, before any discussion of dates? Section V7 answers with the mechanical metrics of the DCMA 14-point model, each with its formula in words, declared population and quoted threshold so it can be contested, plus what a planner asks of any schedule.

IMPLEMENTATION, NOT CERTIFICATION

The thresholds come from a published model and are quoted so they can be discussed. Two of the fourteen metrics cannot be computed from a file and the report says so instead of approximating: the critical path test requires perturbing the schedule, and the critical path length index is 1.0 by construction without a deadline on the final milestone.

CODE	METRIC	POPULATION	THRESHOLD	READS AS
Q1	Missing logic	incomplete, non-milestone	$\leq 5\%$	no predecessor or no successor: delay that does not propagate
Q2	Leads	links	0%	negative lag hides overlap the logic should model
Q3	Lags	links	$\leq 5\%$	positive lag is time with no owner
Q4	Link types	links	$\leq 10\%$ non-FS	every link that is not finish-to-start deserves a reason
Q5	Hard constraints	incomplete	$\leq 5\%$	must start, must finish, finish no later than: a pinned date fights the logic; soft ones are counted, not failed
Q6	High float	incomplete	$\leq 5\%$	over 44 working days: usually a milestone nobody constrained
Q7	Negative float	incomplete	0%	a date the logic cannot reach
Q8	High duration	incomplete, non-milestone	$\leq 5\%$	over 44 working days remaining: too coarse to control
Q9	Invalid dates	all leaves	0%	actual date after the status date
Q10	Unassigned	incomplete with duration	count	nothing to trend
Q11	Missed activities	due by the status date	$\leq 5\%$	did not finish on time
Q12	Critical path test	—	—	not computable from a file
Q13	Critical path length index	—	—	only with a deadline on the final milestone

Q14	Baseline	due by the status	≥ 0.95	finished on time divided by due
	execution index	date		

How to read it

Read Q1, Q2, Q5 and Q7 before any date. A schedule that fails them is not a model of the works, and every date it produces is a number with no basis. Q6 and Q8 say whether it can be controlled. Q11 and Q14 say whether it was met. Beyond the fourteen, the report shows summaries with links, which should not exist, milestones with no deadline, the number of incomplete critical activities, and whether the critical chain reaches the final milestone, which is the test of the network being whole.

ON THE EXAMPLE SCHEDULE

An activity pinned by a must-finish-on constraint appears in Q5. A link with a three-day lag appears in Q3. A summary carrying a link is flagged. The baseline execution index falls below the floor because a due activity did not finish.

Productivity and trend by resource

Section V6 answers, for every activity in progress with a quantity, when it finishes at the actual rate, and what that does to the baseline and the critical path. Everything comes from the file's own assignments; no custom field is required.

Where the quantities live

For a material resource, Project stores the quantity in the work fields, as hours of a duration. An assignment with 10,567 hours of work on a resource labelled m³ is 10,567 m³. The same rule gives executed, remaining, baseline planned and executed per day. An assignment with no resource cannot be trended, and the report counts how many there are.

Three rates, not one

One rate alone is not a fair reading: a cut activity hit by a fortnight of rain shows a rate that says nothing about the crew.

RATE	HOW	WHAT IT ANSWERS
own	executed on this activity divided by working days elapsed since its actual start, on its calendar	what is happening here, rain included
global	everything executed by the resource on all fronts divided by the working days those assignments took	the fair scenario, which dilutes a bad fortnight
recent	the last 30 days of the resource's executed blocks divided by the window's working days	the pace now, not the average of months

The own rate divides by elapsed days, including days when nothing came out. A crew that produced on eight of ten days practised the ten-day rate. The planned rate also comes out, baseline quantity over baseline duration, and two required rates: remaining over working days to the forecast finish, and to the late finish.

Four dates, three verdicts

The remainder runs over each rate, in working days of the activity's calendar from the status date, and lands against four dates: forecast finish, baseline finish, late finish, which the file already carries with the float built in, and the projected finish.

PROJECTED LANDS	VERDICT	MEANING
by the forecast	ahead	nothing to do
after the forecast, by the baseline	reprogram	the forecast is wrong, the baseline is not threatened

after the baseline, by the late finish	baseline delay, float absorbs	no impact on the end date yet; the column says how many float days remain
after the late finish	enters the critical path	on the date shown, before the schedule shows it

ONE CASE READ END TO END

Baseline on 30 September, forecast on the 26th, twenty days of float. If the rate projects 28 September, it is reprogram: the forecast slips, the baseline does not. If it projects 2 October, the baseline slips two days and eighteen days of float remain: no impact on the end date. If it projects more than twenty days beyond the forecast, it enters the critical path, and the date that happens appears in the table.

Thin evidence

A rate built on a few days, or on a tiny share of the planned quantity, projects absurdity with complete ease: a resource at two percent executed "finishes" in 2031. The rate is reported, because it is what the data says, but it is flagged as thin evidence when it has under ten days of data or under two percent of the planned quantity, and the verdict carries the flag. Executed derived from physical percent, rather than posted quantity, is also flagged as inferred. Read those as a question, not a verdict. The thresholds are adjustable through the profile.

Contracted productivity

If the organisation keeps a contracted productivity per crew in a custom field, the profile can point to it and it appears as a comparison column beside the rates. It is an optional layer, never a requirement.

Looking forward and looking back

Section V8 answers when the contract really finishes and what the next weeks demand. Section V9 answers where each milestone's delay came from and by which path. Both use only what the file already carries, and every reading says what it was built on.

Earned Schedule

Classic earned value measures schedule in money, and at the end of the project the indicator goes to 1.0 on a project a year late. Earned Schedule measures schedule in time. Take today's earned value, go up to the planned curve and read the date the curve got there. That date is ES: where the plan said we would be.

QUANTITY	FORMULA	READS AS
ES	date at which cumulative planned equalled today's earned, interpolated inside the month	where the plan said we would be
AT	days from the first planned day to the status date	time elapsed
SV(t)	ES minus AT	days behind the plan; negative is behind
SPI(t)	ES divided by AT	schedule efficiency; 0.90 is ten percent slower than the plan
PD	first planned day to the last baseline finish	planned duration
IEAC(t)	PD divided by SPI(t), as a date from the first planned day	independent finish at this efficiency
TSPI	(PD minus ES) divided by (PD minus AT)	efficiency the remainder needs to finish on the date

TSPI is the meeting number. Above 1.0 the remainder must run faster than the plan. Above 1.10 the literature treats it as unrecoverable, because no works sustains ten percent above plan for long. It turns "we are late" into "we need this much more per week, and that is or is not possible".

THE LIMIT, STATED IN THE REPORT

It is an aggregate, cost-weighted measure, and it does not see the critical path: a mass of non-critical activities late shows behind while the end date holds, and the reverse too. So the independent finish never replaces the schedule's own finish. It sits beside it, and the gap between the two is the conversation. It inherits everything the planned curve inherits: the slot, the earned-value method, execution ahead of the baseline window.

ON THE EXAMPLE SCHEDULE

Earned is far behind planned, and ES falls more than a hundred days before the status date. Schedule efficiency is below 0.2 and TSPI above 1.7: the remainder would need to run seventy percent faster than the plan. The report writes that as a question about the schedule, not as a sentence.

Look-ahead

Four and eight weeks from the status date: what must start and what must finish, by group, weighted by baseline cost; the earning the file plans for the window, pro-rata from the phased curve; and the earning the last two months' practised rate would deliver. One word says whether the planned measurement of the window is reachable.

Rates by group

Percent earned per week to date, from the group's first earning, against the percent per week the remainder needs to land on the group's last baseline finish. The ratio required over practised gives one word per group: on pace, stretch, out of reach.

Milestone bands

The distribution of finish slippage, current finish minus baseline finish in calendar days, across started or due activities in this snapshot, applied to each future milestone's current finish as P50 and P80. Empirical, not a simulation with an invented premise. A series of snapshots replaces it with observed cycle-to-cycle movement.

Rain exposure

Remaining cost spread evenly over each activity's current span from the status date, read against the months its own calendar reserves for bad weather, a month with four or more non-working exceptions. Reported now and again with everything slipped thirty days, because slipping into the reserve is how a small delay becomes a large one. The spread is inferred; the reserve months are measured.

Forensics: driving path per milestone

Where did the delay come from, by which path, and when did it start? From each future milestone the tool walks back through the predecessor that actually drives each start, the one whose link instant plus lag lands latest on the successor's calendar, until it reaches a finished activity, because the past drives nothing, or an open end.

Each node on the chain carries its finish variance against the baseline in working days. The **origin** is the deepest node on the chain still carrying at least five working days of variance: walking back from the milestone, variance is inherited from the driver, so the last node that still has it is where it entered. The delay gets a name and a row number, not an aggregate. The report also says which activities originate delay on the most milestones, because one name can be the root of a dozen slipped dates, whether each

chain reaches started work, and whether it dies on an open end, which is finding Q1 seen from the milestone's side.

NOT A DELAY CLAIM

A contractual claim needs contemporaneous records and a formal method, such as the AACE recommended practices on forensic schedule analysis. This reconstructs the mechanism from the file's dates, links and calendars, with reproducible evidence, to sustain the meeting and the notification. The contractual conversation happens in real time, with each organisation's reality.

ON THE EXAMPLE SCHEDULE

There is a future contractual milestone with twelve working days of variance. The driving chain has three nodes, and the origin is the milestone itself: the variance did not come from any predecessor with delay of its own, which points to a typed date or to a chain broken before it.

Calendar forensics

A weather reserve registered for one year while the activities on that calendar execute in another embeds optimism nothing in the file announces: the month gets its full working days instead of its real ones. Per calendar in use, non-working exceptions per year against activity-days per year, with two flags: years with activities and no reserve while the calendar has reserve in another year, and years with reserve and no activities.

Execution pattern

Out-of-sequence execution by the month the successor started, counting the A1 pairs and the both-complete pairs recorded outside it. And the distribution of start slippage against the baseline across started activities: P20, P50, P80, how many started early, how many late.

With a previous snapshot

Float consumed per activity and per group, total float before minus total float now, in working days: float that disappears while dates hold is delay still invisible. Activities that became critical this cycle. And each milestone's movement between the snapshots, attributed to what moved on its driving chain through the comparator's three signals: execution, replan or moved reference, with the majority reading as the attribution.

Cycle comparison

The single-file review asks whether the schedule holds together inside.

Comparing two asks something else: what moved since the last delivery, and was it the works that moved or the plan?

Input

```
python3 ~/.claude/skills/cronoaudit/scripts/review.py previous.xml current.xml
```

Two snapshots of the same schedule: the previous version and the current one. Not two different schedules, and not a schedule against an application's database. Each schedule compared against itself, which is what makes the reading defensible. With two files come the comparison and the review of the current file, deliberately: comparing two files that do not hold together is comparing two wrong answers.

Each snapshot needs a status date and a saved baseline. Without a baseline there is no planned half and the deviation decomposition has no denominator.

Matching an activity across versions

First by the stable unique identifier, not the row number, which renumbers freely. Failing that, by name plus WBS path. What does not match falls into two classes, and only two: **inserted**, present in the current with no pair in the previous, and **removed**, present in the previous and never matched. A renamed activity that also moved branch is indistinguishable from a removal plus an insertion, and pretending otherwise manufactures a third category that cannot be verified. The report shows the count for the reader to judge.

A FALSE PAIR IS WORSE THAN A MISSED PAIR

A pair that does not match lands in the unmatched list, where somebody looks. A pair matched wrongly produces a date movement, a deviation and a trend entry for two activities that have nothing to do with each other, all plausible. Summaries and milestones stay out of deviation and trend: summaries would double-count their children, and milestones have no duration.

The three signals

SIGNAL	READING
dates moved and actual dates are present	execution
dates moved, no actual dates	replan
baseline dates moved	reference moved

The third is the one to look at first. When the baseline changes between two cycles, the basis of the comparison moved underneath it, and every deviation that straddles the

change is measured against two references. Report it as a finding about the report, not as performance. The second is where the conversation between the parties is decided: a forecast rewritten with no execution behind it is a plan change, legitimate, but it has to be visible as one, otherwise the next cycle starts from a reference nobody agreed to.

The deviation decomposition

For each leaf activity with baseline cost above zero, the contribution is its earned minus planned divided by total baseline cost, in percentage points. The contributions sum to the aggregate, and those that round to zero are dropped, because a list of zeros hides the ones that matter. When the sum does not match the movement, the residue appears as **unexplained**, with the number, instead of vanishing. The denominator is declared: summary total or sum of the leaves.

It is the operational answer to the aggregate problem. A positive aggregate progress can be out-of-sequence work covering activities that should be complete and sit at zero; the decomposition is what lets that be read on the spot.

One function, called twice

Both snapshots' indicators are computed by the same function, called twice, and not by two paths that happen to agree. If the previous cycle's figure were computed differently from the current one, the difference between them would be a reference artefact, and it would read as performance. The two halves are not verified to agree; they are built so they cannot disagree.

What comes out

A cycle report and a JSON, named after both files. Inside: both snapshots' general indicators by the identical rule, the movement in points, the per-activity decomposition that sums to it, the unexplained residue, the trend with the three-signal reading per activity, the moved baselines, the inserted and removed, and the warnings: a baseline that changed slot between cycles, for instance, comes out as a warning.

ON THE EXAMPLE PAIR

Two activities moved dates: one with actual dates, read as execution; another without, read as replan. One had its baseline changed, and the report issues a moved-reference warning before any performance figure. The decomposition has one row, which sums to the movement.

Profile and customisation

Everything an organisation may declare instead of having it coded. Every key has a default that works with no profile at all; a profile only narrows. An organisation's mapping is its own asset and does not belong in the generic method. What is generic is that it can be declared.

Discover before asking

```
python3 ~/.claude/skills/cronoaudit/scripts/discover_fields.py delivery-review.json
```

It lists every custom field that actually carries values, with the type inferred from the values, the fill rate, how many distinct values there are, whether it is a closed set, and a sample. Then it groups them by likely role: discipline, work front, chainage, contractor, phase, justification, quantity, unit. It exists so nobody answers from memory. Asking which field holds the discipline makes a person recall; showing three populated fields with their values makes them recognise. A field that exists and is almost empty comes out separately, as a finding: a justification blank on 96% of activities is a compliance issue, not a field to ignore.

The profile keys

KEY	DEFAULT	EFFECT
organisation	empty	name shown on the report cover
lang	follows the schedule	forces pt or en
ev_method	as the file declares	physical or percent; recorded as forced
baseline_slot	elected by coverage	0 to 10; recorded as forced
threshold_days	30	E and C, in working days of the activity's calendar
tolerance_days	1	G
checks_optional	none	B and F promoted to findings
grouping	wbs	a custom field by alias, name or id
fields.*	none	discipline, service, section, work front, justification, quantity, unit, contracted productivity, regulator code
productivity.*	30 · 10 · 0.02	recent-rate window; thin-evidence thresholds
rain.reserve_days_per_month	4	exceptions that make a month a reserve month
lookahead_weeks	4, 8	the look-ahead windows

theme.*	shipped palette	accent, severity colours, fonts, header text
template	shipped	path to a customised template
sections_hidden	none	section ids to hide, such as v6 or v9

A declared field the file does not have is reported at the top of the report, never silently ignored.

Create and edit

```
python3 scripts/profile_tool.py init --out profile.json --organisation "..."  
python3 scripts/profile_tool.py discover delivery-review.json  
python3 scripts/profile_tool.py set profile.json grouping=DISCIPLINE  
fields.discipline=DISCIPLINE  
python3 scripts/profile_tool.py show profile.json delivery-review.json  
python3 scripts/review.py delivery.xml --profile profile.json
```

The profile is written by the tool, never by hand, and validated on every write. The show command resolves every declared field against the file and says which were found. A filled example ships with the skill, to copy.

The two interviews in the assistant

With the skill in an assistant, say **customise the organisation profile**. It runs discovery first, shows what the file has, and asks one question at a time only about what the file could not settle: which field is the discipline, the section, the justification, the contracted productivity; whether the declared earned-value method is the practised one; whether B and F count as findings; the threshold; the slot, only if the elected one is wrong. Say **customise the report** for colours, fonts, header, language and sections to hide. Deeper changes go in a copy of the template, which takes precedence and survives updates.

The executive synthesis

```
python3 scripts/inject_narrative.py delivery-review.html synthesis.txt --who "your name"
```

Each section's reading is generated by rule. The synthesis is different: a text written by whoever runs the analysis, or by the assistant on request, injected into the report under its own label, with author and time. The report is rebuilt from its own embedded data, so the figures cannot drift from what the text was written against. Inside an assistant, ask for the synthesis and it writes it from the JSON files and injects it; what it writes is interpretation, and is labelled as such.

The JSON files

Every analysis writes its data beside the report. They serve a spreadsheet, a dashboard, a historical series, or an assistant reading and writing. Key names are in English and stable across versions.

Single-file review

–REVIEW	blocking, what prevented the analysis; conventions, threshold, slot and election basis, calendars in use, link types evaluated, lag honoured, ignored pairs; counts, the three counts per finding; findings, the activities per finding with row, name, dates, variance, float; pairs, the explicit A1 and A2 pairs; ignored, the both-complete pairs.
–SCURVE	totals, planned and earned to date, the alternative method, SV and SPI; reconciliation, how many tasks the phasing closes against cost, against the file's planned and against the file's earned, and epoch blocks dropped; periods, the monthly series with cumulatives; by_group, one curve per group; provenance, where each series comes from.
–PRODUCTIVITY	resources, global and recent rates per resource; activities, per activity in progress the three rates, the four dates, the verdict and the thin-evidence and inferred-executed flags; summary, counts per verdict and unassigned assignments.
–QUALITY	metrics, the fourteen with count, population, share, threshold, status and items; indices, baseline execution index, critical activities, whether the critical chain reaches the final milestone; qualitative, summaries with links and milestones with no deadline; soft_constraints, the soft constraints counted.
–FORECAST	earned_schedule, every quantity from chapter 8; lookahead, one entry per window; rates_by_group; milestone_bands, sample, P50, P80 and one record per milestone; rain_exposure, remaining in reserve now and with thirty days of slip.
–FORENSICS	milestones, each future milestone's driving chain with its origin; summary, recurring origins and open chains; calendars, reserve per year against activity-days, with flags; execution, out-of-sequence by month and start slippage; cycle, with two files, float consumed, became critical, attributed milestone movement.
–READINGS	readings, each section's rule-built sentences, in the report's language, exactly as the HTML shows them; language, the detected language, its score and confidence. It is the file an assistant quotes instead of composing readings of its own.

Cycle comparison

–CYCLE	previous and current, each snapshot's general indicators by the same rule; movement_points, decomposition_sums_to_points and movement_unexplained_points; trend, per activity, days moved and the three-signal reading; baseline_moved; unmatched, inserted and removed; decomposition; warnings; conventions, matching, population, denominator.
--------	---

Two identifiers, always together

In the JSON files each activity carries the stable unique identifier, which is what joins data across versions, and the visible row number, which is the only one a person finds on screen. In the report only the second appears. A finding list in unique identifiers is unusable in a meeting; a list in row numbers is unusable for joining versions. So both travel together in the data.

Re-reading the report as data

The HTML carries its own data embedded in a block at the end of the file. That is what allows rebuilding it to inject the synthesis, and what lets one cycle's report serve as input to the next with no database: the historical series travels inside the generated file. The cost is that the chain breaks if a cycle's file is lost, so the file is the asset. Keep it.

Limits, labels and what the tool does not decide

A defensible report says what it does not know. This chapter gathers the tool's limits, the labels it uses to separate measurement from inference, and the judgements that stay with the person.

The three labels

LABEL	MEANS	EXAMPLE
measured	read directly from the file, reproducible by anyone with the same file	the A1 count, the planned curve, the number of calendars in use
inferred	derived by a declared rule, from measured data	executed derived from physical percent; the even spread of the remainder in the rain exposure
reported	comes from a documented record, not from this analysis's measurement	production-use results cited in the method references

The label travels with the figure in the report and the JSON files, and an inferred figure is never presented as measured.

What the tool refuses to do

- **Assume today's date** when the file has no status date. The analysis stops and says why.
- **Invent the planned half** when no baseline slot carries cost. Without a baseline there is no adherence.
- **Approximate the two quality metrics** a file cannot answer. Q12 and Q13 come out as not computable.
- **Guess a distribution.** The milestone band is empirical, from observed slippage; the rain exposure uses an even spread and says so.
- **Match activities by similarity.** Only stable identifier or name plus path. The rest is inserted or removed.
- **Replace the schedule's finish** with Earned Schedule's independent finish. The two sit side by side.

Judgements that stay with the person

G is a thermometer: use it to choose what to inspect, never report it alone. The physical percent may have been typed, and everything computed from it inherits that; the report says which field it used. The baseline slot is elected from the data, because the field that should declare it is not dependable; check it with whoever owns the schedule. If the calendar table shows a shift or week different from what the works practise, every date

inherits the error. A thin-evidence rate is a question. A forensic origin that lands on the milestone itself points to a typed date or a broken chain, and someone has to open the file.

Formats and scope

MS Project XML is the measured format. Reading .mpp and Primavera P6 through a Java library is optional and unverified. The tool does not produce a contractual claim, does not certify against a standard, and does not replace the meeting: it delivers what the meeting needs so it does not turn into an argument about the arithmetic.

Frequently asked questions

The report flags zero on everything in a file I know has problems

Look at the population line in the conventions. Summary-only files, or files with everything marked external, have no leaves to test. And run the internal test: if the positive fixture does not fire, the installation is broken.

cronoaudit's count does not match my macro's

First check the convention: distinct activities, as successors or as predecessors give three numbers for the same problem, and the report shows all three. Then check the baseline slot and the threshold. Finally, the both-complete pairs: cronoaudit leaves them out of A1 and records them separately. The macro implementation follows the same specification, and on the same file the counts have to match, convention by convention.

Earned value does not match Project

The reconciliation at the foot of the S-curve says on how many tasks it matches and classifies those that do not. The common cause is execution ahead of the baseline window: Project only credits the phased cost when the status date reaches the window, and the method credits cost times physical at once. Both are right against their own instant. If the difference is something else, the slot may be wrong.

Why is the report in English when the schedule is mine

The language is detected from the file's content: activity names, calendars, title. A schedule with English names produces an English report. Force it with the language option or through the profile.

Can I change the 30-day threshold

Yes, through the command-line option or the profile. The report then declares the new value in the conventions. It counts in working days of the activity's calendar, so thirty working days on a six-day week are about 35 calendar days.

E flagged activities that were only reprogrammed

That is what E looks for: finish pulled earlier with no execution. Reprogramming earlier can be legitimate, but it needs a reason, and E is the list of questions. If the reason exists, it gets recorded; if not, the forecast is optimistic.

Does sensitive data leave the machine

No. The tool does not access the network. The report is a local file with no external reference. An assistant only sees what it is shown, and the synthesis is the only part it writes.

How do I update

Run the installer again. It replaces the skill and keeps a customised template, if there is one. The organisation profile stays where it was saved and is not touched.

Does it work with Primavera P6

The method transfers, but nothing was measured in that format. Reading through a Java library exists as an optional path and the result should be treated as unverified.

Glossary

The terms as the report uses them, in English, with the Portuguese equivalent.

Baseline *linha de base*

Copy of the plan saved at an instant, against which variance is measured.

Baseline slot *gaveta de linha de base*

One of the eleven copies of the plan Project keeps, 0 to 10. Elected by cost coverage.

Counting convention *convenção de contagem*

How activities are counted in a network finding: distinct, as successors or as predecessors.

Critical path *caminho crítico*

Chain of activities with zero total float, from start to finish. Any delay on it moves the end.

Deadline *data limite*

Milestone field that reveals slippage without moving any date. Not a constraint.

Decomposition *decomposição*

Each activity's contribution to the aggregate deviation, in percentage points, summing to the total.

Driving path *caminho condutor*

Chain of predecessors that actually sets a milestone's date, chosen link by link by the one that lands latest.

Earned Schedule *ES*

Date at which the planned curve reached today's earned value. Schedule measured in time.

Finish variance *variação de término*

Current finish minus baseline finish, in working days of the activity's calendar.

FS, SS, FF, SF *TI, II, TT, IT*

Finish-to-start, start-to-start, finish-to-finish, start-to-finish: the four link types.

Lag and lead *lag, lead*

Mandatory wait and permitted overlap on a link. A lead is negative lag.

Leaf activity *atividade folha*

Activity with no children in the WBS. The only one the analysis counts; summaries are aggregations of it.

Look-ahead

What must start and finish in the coming weeks, and whether the pace reaches it.

Origin *origem*

Deepest node on the driving path still carrying variance. Where the delay entered.

Physical percent complete *percentual fisico*

How much of the work is out, declared. Distinct from duration percent.

Planned and earned value *previsto e realizado*

What should have been earned by the status date, and what was earned.

Profile *perfil*

File that declares an organisation's conventions and fields instead of coding them.

Reference *referência, datum*

Slot, instant, population and unit against which a figure is measured. Two halves against different references do not compare.

S-curve *curva S*

Cumulative planned and earned over time. The gap between the curves is the delay in money.

SPI *IDP*

Schedule performance index: earned divided by planned, in money.

Status date *data de status*

Date up to which progress is valid. Every planned against actual is measured at it.

Thin evidence *evidência fina*

Rate built on under ten days or under two percent of the planned quantity. Reported with a flag.

Total float *folga total*

How much an activity can slip without moving the project finish.

Trend, forecast dates *tendência*

Current dates of the live schedule, computed by the network from what was reported.

TSPI

Efficiency the remainder needs to finish on the planned date. Above 1.10, treated as unrecoverable.

Weather reserve *reserva de improdutividade*

Non-working days registered in the calendar for rain. A month with four or more is a reserve month.